

Paso 1 - El primer programa | Ejercicios

Trabaja en tu carpeta, la misma donde escribiste `Paso01Hola.java`. Cada ejercicio es un **archivo nuevo**: acuérdate de que el archivo se llama igual que la clase.

```
javac Archivo.java
java Archivo
```

► **Plantillas:** [plantillas/](#) — un `.java` por ejercicio, con el enunciado, la salida esperada y los `// TODO` donde va tu código. Si las usas, el archivo ya viene creado (`B1_1_Ficha.java`) y **ese** nombre manda sobre el que diga el enunciado.

El `Practica.java` del B1.3 ya viene roto: ese es el ejercicio.

✓ **Solucionario:** [soluciones/](#) — el código de cada ejercicio, la salida real y por qué se resuelve así. Ábrelo **después** de intentarlo.

Cómo se lee cada ejercicio: el título dice qué hay que lograr; el bloque de abajo es la **salida esperada**, o sea lo que tiene que aparecer en tu pantalla (en los programas que preguntan cosas, lo que va después de la pregunta es lo que **escribe el usuario**); las citas como esta son pistas. Si no sabes por dónde partir, abre la plantilla: los `// TODO` numerados son los pasos, en orden. Compila y ejecuta después de cada uno.

B1.1 - Tu ficha

Qué practicas: crear un archivo `.java` desde cero, hacer que el nombre de la clase coincida con el del archivo, e imprimir con `println`.

Escribe `Ficha.java` que imprima tres líneas: tu nombre, tu carrera y tu sección.

```
Ana Perez
Ingenieria en Informatica
Seccion 003D
```

Si escribes tildes o ñ y salen como Ã±, déjalo sin tildes: es el idioma de la terminal, no un error tuyo.

B1.2 - Cambiarle el nombre a la clase

Qué practicas: la regla de que el archivo y la clase pública se llaman igual, y a leer el mensaje completo de `javac` cuando se rompe.

Trabaja sobre el archivo del B1.1 (o sobre la plantilla, que ya trae un programa listo para renombrar).

1. Agrégale `Alumno` al final del nombre de **la clase**, en la línea `public class ...`, y **no toques el nombre del archivo**. Compila.
2. Copia el error completo en un comentario dentro del archivo.
3. Ahora renombra también el archivo, para que vuelva a llamarse igual que la clase, y compila de nuevo.
4. Anota, en otro comentario: para renombrar **una** clase, ¿cuántas cosas tuviste que cambiar?

El error del punto 1 es el mismo del último "Rómpelo" del paso 1. Aquí lo importante es el punto 4.

B1.3 - Caza de errores

Qué practicas: identificar y corregir, de a uno, los errores de compilación más comunes: un `;` que falta, una mayúscula equivocada y una llave sin cerrar.

Copia este programa **tal cual** en `Practica.java`. Tiene cuatro errores.

```
public class Practica
{
    public static void main(String[] args) {
        System.out.println("Primera linea")
        system.out.println("Segunda linea");
    }
}
```

Arrégloslos **de a uno**, compilando cada vez: `javac` no los muestra todos juntos, así que van apareciendo a medida que arreglas los anteriores.

Anota en una lista: el mensaje que salió y qué le faltaba al programa.

Los cuatro errores son: una llave que falta, un `;` que falta, una mayúscula y una llave sin cerrar. El orden en que aparecen **no** es ese.

B1.4 - Dos programas en la misma carpeta

Qué practicas: compilar varias clases de una vez con `javac *.java`, y que a `java` se le pasa el nombre de la clase, no el del archivo.

1. Crea `Uno.java` y `Dos.java`. Cada uno con su `main`, que imprima "Soy el uno" y "Soy el dos".
2. Compila los dos de una vez con `javac *.java`.

3. Ejecuta `java Uno` y después `java Dos`.

```
$ javac *.java
$ java Uno
Soy el uno
$ java Dos
Soy el dos
```

Preguntas:

- ¿Cuántos archivos `.class` hay ahora en la carpeta?
- ¿Cómo sabe `java` cuál de los dos programas ejecutar?
- ¿Qué pasa si escribes `java Uno.class`? ¿Y `java uno`?

B1.5 - Desafío: una sola letra

Qué practicas: distinguir un error de símbolo (`cannot find symbol`) de un error de sintaxis, y leer el mensaje completo de `javac` antes de tocar el código.

Parte de un programa que compile y **no imprima nada**: una clase con un `main` vacío ya cumple. Compíllalo para comprobarlo.

Ahora hazle **estas tres mutaciones, una a la vez** (deshaz cada una antes de la siguiente) y anota el mensaje completo de `javac`:

Mutación	Queda
a	<code>public static void main(Strin[] args)</code>
b	<code>public static voi main(String[] args)</code>
c	<code>publi class B1_5_Desafio</code>

Después responde, mirando tus tres anotaciones:

1. Dos de los mensajes te dicen **exactamente** qué palabra está mal escrita. ¿Cuáles son, y en qué parte del mensaje aparece esa palabra?
2. El tercero no nombra la palabra: solo dice qué esperaba encontrar. ¿Cuál es?
3. En ese tercer caso, ¿por qué Java cree que falta una clase entera, si tú cambiaste **una sola letra**?

Objetivo del ejercicio: que empieces a leer el mensaje de error **como una pista de dónde mirar**, no como un castigo. El error casi nunca está donde `javac` dice: está ahí **o un poco antes**.

Cuando termines, sigue con el [paso 2](#).